

Region Resonances

Methods Primer

A Guide to the Statistical and Computational Methods

Jure Skarabot · May 2026

Companion documents: Summary · Technical Appendix · Methods Primer · Data Appendix · Region Reference (Identity)

Vinotheca · published under CC BY-NC 4.0

Introduction

Region Resonances is a query tool that takes a piece of text from the user — a word, phrase, or short passage — and returns the wine regions whose written character is most similar to it. The procedure has three stages: convert text to numerical vectors, compute similarities, sort. This primer explains each stage in plain language. The treatment assumes comfort with basic mathematics (vectors, angles, distances) but not prior exposure to natural-language processing or information retrieval. For full mathematical detail, see the Technical Appendix.

1. Semantic Embeddings

What an embedding is. An embedding is a list of numbers that represents the meaning of a piece of text. Computers cannot work with text directly; they need numbers. Embeddings give them a way to convert text into numbers in a manner that preserves meaning: passages with similar meanings get similar embeddings, passages with different meanings get different embeddings.

The crucial property of a semantic embedding is that it abstracts away from surface vocabulary. Two passages can share no common words and still receive nearly identical embeddings if their meaning is close. Conversely, two passages can share many common words — think of two sentences differing only by a negation — and receive quite different embeddings. The embedding captures meaning at a level above the words themselves.

How they are produced. The embedding model used here is OpenAI’s `text-embedding-3-small`, a large neural language model. The model has been trained on a vast corpus of text from the internet and books; through that training, it has learned that certain patterns of words tend to mean similar things, and it has learned to encode those patterns into similar numerical vectors. We do not need to know exactly how the model represents meaning internally; we need only know that, given an input passage, it produces a 1,536-dimensional vector, and that semantically similar passages reliably produce similar vectors.

What dimension means. Each embedding has 1,536 numbers because the model was designed that way. Each of those 1,536 dimensions captures a fraction of the semantic content of the

passage. There is no human-interpretable dimension — you cannot point to dimension 137 and say “this is the dimension that measures sadness” — but in the aggregate, the 1,536 numbers carry enough information to distinguish “solitude” from “community”, “volcanic resilience” from “Mediterranean ease”. Each number is a fragment of meaning; the whole vector is the passage’s location in a high-dimensional space of meanings.

For Region Resonances. Each of the 59 region passages (a one-word metaphor concatenated with a 280–320-word identity narrative) has been passed through the OpenAI model once, producing 59 vectors of 1,536 numbers each. These are computed in advance and stored. At query time, the user’s input is passed through the same model, producing one more vector of 1,536 numbers. The matching procedure then compares this query vector to each of the 59 region vectors.

2. Unit Normalisation

What it does. Unit normalisation rescales every vector to have length 1. If a vector originally has length 5, divide every entry by 5; the result has length 1. Geometrically, this places every vector on the surface of a sphere of radius 1 centred at the origin — the *unit sphere*. Two passages with similar meaning produce vectors that point in nearly the same direction; after normalisation, they sit close together on the sphere’s surface.

Why it matters. The OpenAI embedding model produces vectors that are already unit-normalised. This means we can ignore lengths entirely and work only with directions. Two embeddings “point the same way” or “point different ways” — the magnitude carries no information. This simplifies similarity computation considerably (see next section).

The unit sphere as a meaning space. Once all 59 region vectors and the query vector lie on the unit sphere, the matching question takes on a simple geometric form: *which region points are closest to the query point on the surface of the sphere?* Closest on the sphere can be measured several ways — by the angle between two points, by the straight-line distance through the sphere’s interior, or by the cosine of the angle — but all three methods agree on which point is closer. We use the cosine, for reasons explained below.

3. Cosine Similarity

What it does. Cosine similarity measures how aligned two vectors are. Given two vectors $\mathbf{x}$ and $\mathbf{y}$, the cosine similarity is the cosine of the angle between them. It ranges from -1 (vectors point in opposite directions) through 0 (vectors are perpendicular, unrelated) to $+1$ (vectors point in the same direction).

The formula. For two vectors $\mathbf{x}, \mathbf{y} \in \mathbb{R}^d$,

$$\cos(\mathbf{x}, \mathbf{y}) = \frac{\langle \mathbf{x}, \mathbf{y} \rangle}{\|\mathbf{x}\|_2 \|\mathbf{y}\|_2},$$

where $\langle \mathbf{x}, \mathbf{y} \rangle = x_1y_1 + x_2y_2 + \cdots + x_dy_d$ is the inner product (the sum of element-wise products).

The unit-sphere simplification. Because all vectors are unit-normalised, the denominator simplifies to $1 \cdot 1 = 1$, and the formula collapses to just the inner product:

$$\cos(\mathbf{r}_i, \mathbf{q}) = \langle \mathbf{r}_i, \mathbf{q} \rangle.$$

On the unit sphere, cosine similarity is just the inner product, no division required. This is the central computational identity of the Region Resonances system: every similarity measurement is one inner-product calculation.

Why cosine, not Euclidean? Three reasons. First, on the unit sphere, cosine similarity, Euclidean (chord) distance, and angular (great-circle) distance all give the same ranking — so the choice among them is one of convenience. Second, cosine similarity is bounded in $[-1, +1]$, dimensionless, and easy to interpret as “angular alignment”. Third, in high-dimensional spaces, Euclidean distances tend to concentrate around a similar value (a phenomenon known as the curse of dimensionality), making them harder to rank cleanly; cosine similarity is more numerically robust in this setting.

For Region Resonances. The system computes the cosine similarity (= inner product) between the query vector and each of the 59 region vectors. The result is a list of 59 numbers, each in the range $[-1, +1]$. The region with the highest similarity is the closest match; the region with the lowest similarity is the most distant.

4. Nearest-Neighbour Search

What it does. Nearest-neighbour search finds, among a fixed set of points in a vector space, the one closest to a given query point. In its basic form (used here), it sorts all candidates by similarity and returns the top k . More elaborate variants use indexing structures (k-d trees, ball trees, locality-sensitive hashing) to accelerate retrieval over very large datasets, but for $n = 59$ points the brute-force approach is faster and simpler.

The procedure. Given the similarity vector $\mathbf{s}$ of length 59 (one cosine value per region), the system sorts the 59 indices in descending order of similarity and takes the first $k = 5$. These five regions are returned to the user. The remaining 54 are accessible through a “Show all 59 ranked” disclosure.

Why $k = 5$? A pragmatic choice. The top five matches typically span enough of the relevant semantic neighbourhood to convey the structure of the user’s query, while remaining short enough to read at a glance. Returning fewer than five would suppress informative second-tier matches; returning more than five would dilute the signal with regions that are progressively less semantically relevant. The Region Resonances tool exposes the full 59-region ranking on request, so the choice of $k = 5$ is a default presentation, not a hard cutoff.

5. Display Transformation: Clip-and-Rescale

The problem. Cosine similarity is mathematically bounded in $[-1, +1]$, but on this corpus the values actually observed lie in a much narrower interval. Across a calibration test set of 60 queries, observed similarities fall almost entirely within $[0.15, 0.65]$. The reason is that any two passages of English share a substantial baseline of common semantic structure; the embedding model encodes this shared structure into a common region of the sphere, and the discriminating signal lives in a narrow band above the baseline.

This creates a presentation problem. A score of 0.62 sounds modest but is in fact a strong match. A score of 0.31 sounds low but is the typical similarity for an unrelated query. The differences that matter most — between the top match and the tenth match — often span only a few hundredths of a unit.

The solution. A piecewise-linear transformation maps the empirical operating range $[s_{\min}, s_{\max}] = [0.15, 0.70]$ onto $[0, 100]$:

$$\tilde{s}_i = \text{clip}\left(\frac{s_i - s_{\min}}{s_{\max} - s_{\min}}, 0, 1\right) \times 100.$$

The function $\text{clip}(t, 0, 1) = \min(1, \max(0, t))$ ensures the result stays in $[0, 1]$ before being multiplied by 100. The constants $s_{\min}$ and $s_{\max}$ are calibrated — chosen empirically so that the operating range of the cosine corresponds to the operating range of the display.

Three properties. The transformation is bounded (results always lie in $[0, 100]$, fitting on a percentage scale). It is monotonically increasing on the operating range (the displayed ranking is always identical to the underlying cosine ranking). And it is affine on the operating range (a difference of 10 in the displayed score corresponds to a fixed difference in the underlying cosine, regardless of where on the scale that difference falls). Two regions whose displayed scores differ by 10 are equally distinguishable in the underlying space whether the scores are 40 and 50 or 80 and 90.

Why not softmax? Softmax is a common alternative for displaying similarity scores as percentages:

$$\pi_i = \frac{\exp(\beta s_i)}{\sum_j \exp(\beta s_j)}.$$

It preserves the ranking but has two drawbacks for this application. First, the temperature parameter β is arbitrary; different values produce dramatically different displayed probabilities for the same underlying ranking. Second, softmax is multiplicative in the score gap: small cosine differences become large probability differences, exaggerating distinctions between near-tied results and giving users a false sense of confidence in the top match. Clip-and-rescale, by contrast, is honest about how close near-tied results actually are.

6. The Pipeline in Summary

The methods chain together into a five-stage pipeline. (1) *Embedding*: each region’s passage is converted to a 1,536-dimensional vector via the OpenAI model; this is done once at build time. (2) *Unit normalisation*: every region vector is rescaled to length 1, placing it on the unit sphere. (3) *Query embedding*: at runtime, the user’s query is converted to a vector through the same model. (4) *Cosine similarity (= inner product on the unit sphere)*: 59 inner products are computed in one matrix–vector multiplication. (5) *Top-k selection and display transformation*: the 59 similarities are sorted in descending order; the top 5 are returned with their cosine scores transformed via clip-and-rescale to the $[0, 100]$ display scale.

The result is a ranked list of five regions whose written character most resembles the query, accompanied by display scores that convey the strength of each match without overstating confidence.

References

- Mikolov, T., Chen, K., Corrado, G. & Dean, J. (2013). Efficient estimation of word representations in vector space. *arXiv:1301.3781*.
- OpenAI (2024). New embedding models and API updates. Technical announcement covering `text-embedding-3-small` and `text-embedding-3-large`.

- Reimers, N. & Gurevych, I. (2019). Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Proceedings of EMNLP-IJCNLP*, 3982–3992.
- Salton, G. & McGill, M. J. (1983). *Introduction to Modern Information Retrieval*. McGraw-Hill.
- Vaswani, A. et al. (2017). Attention is all you need. In *Advances in Neural Information Processing Systems* 30.